Seminar I

RANCANG BANGUN APLIKASI PRESENSI KARYAWAN BERBASIS MOBILE MENGGUNAKAN ARSITEKTUR OFFLINE-FIRST DENGAN FLUTTER PADA PT SEMERU AGUNG MANDIRI

Oleh
ABD. ROUF
223140118

Pembimbing Utama	: Nama Dosen Pembimbing 1, S.Kom., M.Kom.
Pembimbing Pertama	: Nama Dosen Pembimbing 2, S.Kom., M.Kom.
Penguji	: 1.Nama Dosen Penguji 1, S.Kom., M.Kom. 2.Nama Dosen Penguji 2, S.Kom., M.Kom.

INSTITUT TEKNOLOGI DAN BISNIS WIDYA GAMA
LUMAJANG TAHUN AJARAN
2025/2026

DAFTAR ISI

DAFTAR ISI i

DAFTAR TABEL iii

DAFTAR GAMBAR iv

BAB I PENDAHULUAN 1

1.1 Latar Belakang Masalah 1

1.2 Rumusan Masalah 3

1.3 Batasan Masalah 3

1.4 Tujuan Penelitian 4

1.5 Manfaat Penelitian 4

BAB II TINJAUAN PUSTAKA 5

2.1 Tinjauan Teoretis Arsitektur Offline-First 5

2.2 Analisis Infrastruktur Penyimpanan Data: On-Premise vs Cloud Storage 6

2.3 Kajian Komparatif Solusi Penyimpanan Lokal Perangkat Seluler . . . 6

2.4 Penerapan Clean Architecture pada Framework Flutter 7

2.5 Flutter dan Dart 8

2.6 PocketBase sebagai Backend-as-a-Service 8

2.7 GetX sebagai State Management 9

2.8 Sistem Presensi dan Validasi Geofencing 9

2.9 Metode Pengembangan Agile Scrum 10

2.10 Penelitian Terdahulu 10

BAB III METODOLOGI PENELITIAN DAN PERANCANGAN SISTEM 13

3.1 Teknik Pengumpulan Data 13

3.2 Spesifikasi Kebutuhan Sistem 13

3.3 Kebutuhan Non-Fungsional 13

3.4 Metodologi Pengembangan Scrum dan Perencanaan Sprint 14

3.5 Perancangan Skema Database Isar 15

3.6 Algoritma Validasi Geofencing dan Sinkronisasi Data 16
3.6.1 Validasi Jarak Geofencing Lokal 16
3.6.2 Mekanisme Sinkronisasi Asinkron 17
3.7 Arsitektur Infrastruktur Sistem 17
3.8 Perancangan Antarmuka (Wireframe) 17

DAFTAR PUSTAKA 17

DAFTAR TABEL

1.1 Perbandingan Karakteristik Paradigma Online-First dan Offline-First . 2

2.1 Perbandingan Infrastruktur On-Premise dan Cloud Storage 6
2.2 Perbandingan Solusi Database Lokal 7
2.3 Matriks Perbandingan Penelitian Terdahulu 11

3.1 Kebutuhan Fungsional Sistem 14
3.2 Perencanaan Sprint 15

DAFTAR GAMBAR

BAB I
PENDAHULUAN

1.1 Latar Belakang Masalah

Kehadiran dan kedisiplinan karyawan merupakan salah satu pilar utama yang menentukan kelancaran operasional dan produktivitas suatu organisasi (Mekari Talenta 2024). Pengelolaan data kehadiran yang akurat sangat dibutuhkan oleh divisi Sumber Daya Manusia untuk memproses penggajian, mengevaluasi kinerja, serta menyusun perencanaan tenaga kerja secara strategis. Meskipun demikian, pencatatan presensi tradisional yang mengandalkan tanda tangan manual atau mesin pemindai sidik jari fisik di kantor sering kali tidak efisien, rentan terhadap kecurangan pertukaran kartu, dan tidak fleksibel bagi karyawan yang bekerja di lapangan (HashMicro 2024).

Sebagai respons terhadap keterbatasan sistem fisik, banyak organisasi beralih ke aplikasi presensi berbasis seluler (*mobile attendance*). Sebagian besar solusi komersial saat ini menggunakan paradigma *online-first*, di mana setiap kali karyawan melakukan pengiriman data kehadiran, aplikasi harus langsung terhubung ke *server awan* (*cloud server*) melalui REST API. Namun, dalam implementasi praktis di lapangan, ketergantungan mutlak pada konektivitas internet melahirkan masalah teknis kritis yang dikenal sebagai *The Basement Problem* (Stackademic 2024).

Kondisi *The Basement Problem* terjadi ketika pengguna berada di lingkungan dengan sinyal seluler yang lemah, tidak stabil, atau bahkan terputus sama sekali, seperti di dalam *basement* gedung, terowongan, gudang logistik pinggiran kota, area proyek konstruksi, atau klinik pedesaan. Pada aplikasi berbasis *online-first*, ketiadaan koneksi internet akan langsung menyebabkan kegagalan fungsi aplikasi, yang ditandai dengan pemutar pemuatan (*loading spinner*) tanpa batas waktu, pesan kesalahan koneksi, kehilangan data transaksi, atau bahkan aplikasi keluar secara paksa (*crash*) (SmartTech Devs 2024). Kegagalan sistem ini tidak hanya memicu frustrasi bagi karyawan, tetapi juga mengakibatkan hilangnya data kehadiran yang valid, sehingga menghambat proses administrasi internal organisasi (Politeknik Negeri Bengkalis 2024).

Studi kasus pada PT Semeru Agung Mandiri menunjukkan bahwa fluktuasi

koneksi internet di lapangan menjadi kendala utama yang merusak akurasi pelaporan kehadiran karyawan. Kegagalan sistem dalam merekam kehadiran secara langsung menurunkan produktivitas dan menciptakan ketidakpastian administratif. Solusi untuk mengatasi tantangan ini adalah dengan mengubah paradigma arsitektur aplikasi dari *online-first* menjadi *offline-first* (ResearchGate 2024).

Arsitektur *offline-first* menempatkan *database* lokal pada perangkat sebagai sumber kebenaran tunggal (*single source of truth*) untuk seluruh operasi baca dan tulis data. Ketika karyawan menekan tombol presensi di area luring, aplikasi secara instan menyimpan transaksi tersebut ke dalam basis data lokal perangkat tanpa memedulikan status jaringan saat itu. Transaksi presensi terekam tanpa latensi jaringan, sehingga memberikan pengalaman pengguna yang sangat responsif. Sinkronisasi data ke server pusat dilakukan di latar belakang secara oportunistik hanya ketika perangkat kembali mendeteksi adanya koneksi internet yang stabil.

Tabel 1.1: Perbandingan Karakteristik Paradigma Online-First dan Offline-First

Karakteristik	**Paradigma Online-First**	**Paradigma Offline-First**
Sumber Kebenaran Utama	Database Server Awan (*Cloud Server*)	Database Lokal Perangkat Mobile
Ketergantungan Jaringan	Mutlak; aplikasi lumpuh tanpa internet	Opsional; fungsi inti berjalan luring penuh
Latensi Operasi UI	Bergantung pada kecepatan internet	Instan (menggunakan memori lokal)
Beban API Server	Tinggi; setiap ketukan memicu panggilan API	Rendah; data dikompresi dan dikirim berkala
Konsumsi Daya Baterai	Boros akibat kegagalan pengulangan API	Efisien; menggunakan strategi *exponential backoff*
Resiko Kehilangan Data	Tinggi jika jaringan terputus saat transaksi	Sangat rendah; data dijamin tersimpan di lokal

Penelitian ini memfokuskan pada rancang bangun aplikasi presensi karyawan berbasis mobile yang berjalan dengan arsitektur *offline-first* menggunakan kerangka kerja (*framework*) Flutter dan *database* NoSQL Isar. Flutter memfasilitasi pembuatan antarmuka lintas platform (Android dan iOS) yang berkinerja tinggi dengan basis kode tunggal. Sementara Isar dipilih karena kecepatannya yang luar biasa dalam memproses kueri lokal, struktur data yang aman secara tipe (*type-safe*), serta dukungan bawaan terhadap kueri reaktif (*reactive stream query*) yang menyederhanakan pembaruan

antarmuka secara otomatis ketika terjadi sinkronisasi data latar belakang.

1.2 Rumusan Masalah

Berdasarkan latar belakang masalah yang telah diuraikan, maka rumusan masalah dalam penelitian ini adalah:

1. Bagaimana merancang arsitektur *offline-first* pada aplikasi presensi berbasis mobile menggunakan kerangka kerja Flutter dengan database lokal Isar?
2. Bagaimana mekanisme sinkronisasi data presensi yang konsisten dari Isar (lokal) ke PocketBase (server) saat koneksi internet kembali tersedia?
3. Bagaimana memvalidasi lokasi presensi karyawan menggunakan GPS Geofencing (formula Haversine) secara lokal tanpa koneksi internet?

1.3 Batasan Masalah

Agar penelitian ini lebih terarah dan tidak menyimpang dari tujuan awal, maka diberikan batasan-batasan sebagai berikut:

1. Aplikasi dikembangkan menggunakan framework Flutter dengan target sistem operasi Android.
2. Penyimpanan data lokal pada perangkat menggunakan Isar Database, sedangkan backend server menggunakan PocketBase.
3. Fitur utama yang dikembangkan mencakup presensi masuk dan pulang, validasi geofencing, riwayat kehadiran, dan sinkronisasi data otomatis.
4. Penelitian ini tidak membahas sistem penggajian (*payroll*) secara mendalam.
5. Pengujian aplikasi menggunakan metode *Black-box Testing*.
6. Studi kasus implementasi sistem dilakukan pada PT Semeru Agung Mandiri, Lumajang.

1.4 Tujuan Penelitian

Sejalan dengan rumusan masalah di atas, tujuan dari penelitian ini adalah:

1. Merancang dan membangun aplikasi presensi karyawan dengan arsitektur *offline-first* menggunakan Flutter dan Isar yang handal di area minim sinyal internet.
2. Menerapkan mekanisme sinkronisasi data dari perangkat lokal ke server PocketBase secara otomatis saat koneksi internet pulih.
3. Mengimplementasikan dan menguji validasi lokasi menggunakan metode GPS Geofencing (Haversine) yang dapat berjalan tanpa koneksi internet.

1.5 Manfaat Penelitian

Manfaat yang diharapkan dari penelitian ini adalah:

1. Memastikan karyawan dapat melakukan presensi kapan saja dan di mana saja tanpa terhambat oleh masalah konektivitas jaringan (The Basement Problem).
2. Meningkatkan efisiensi konsumsi baterai perangkat karyawan karena pengurangan kegagalan *request* ke API server.
3. Memudahkan tim HRD PT Semeru Agung Mandiri dalam merekapitulasi data kehadiran yang lebih akurat tanpa adanya data yang hilang akibat gangguan jaringan saat proses transmisi.

BAB II
TINJAUAN PUSTAKA

2.1 Tinjauan Teoretis Arsitektur Offline-First

Arsitektur *offline-first* merupakan sebuah paradigma perancangan perangkat lunak di mana konektivitas jaringan tidak lagi diperlakukan sebagai prasyarat utama, melainkan sebagai mekanisme sinkronisasi data sekunder (ResearchGate 2024). Berdasarkan Teorema CAP (*Consistency, Availability, Partition Tolerance*), sebuah sistem penyimpanan data terdistribusi hanya dapat menjamin maksimal dua dari tiga properti tersebut secara bersamaan. Pada perangkat seluler yang sering mengalami gangguan sinyal (toleransi partisi), arsitektur *offline-first* memilih untuk mengorbankan konsistensi seketika (*immediate consistency*) demi mempertahankan ketersediaan tinggi, yang mengarah pada model konsistensi akhir (*eventual consistency*) (Nair 2020).

Dalam model konsistensi akhir, data pada *client* dan *server* akan mengalami perbedaan (*drift*) selama masa luring, namun dijamin akan mencapai keadaan yang sama setelah proses sinkronisasi berhasil diselesaikan. Penyelarasan keadaan ini membutuhkan struktur data khusus seperti *Conflict-Free Replicated Data Types* (CRDTs) atau algoritma penyelarasan semantik guna menghindari benturan data. CRDT yang umum digunakan meliputi G-Counter untuk akumulasi peristiwa monotonik, *Observed-Remove Set* (OR-Set) untuk koleksi data dinamis, dan *Last-Writer-Wins Register* (LWW-Register) untuk memperbarui variabel keadaan berdasarkan tanda waktu (*timestamp*) terbaru (Nair 2020).

Keunggulan komparatif strategi sinkronisasi dianalisis secara empiris oleh Nair et al. (2020) yang membandingkan efisiensi konsumsi *bandwidth* dan tingkat benturan data antara tiga metode utama, yaitu pengiriman *snapshot* penuh (*full-snapshot sync*), pencatatan log operasi (*operation-log sync*), dan sinkronisasi delta (*delta synchronization*). Hasil penelitian tersebut menunjukkan bahwa sinkronisasi delta, yang hanya mengirimkan bagian data yang mengalami modifikasi sejak waktu sinkronisasi terakhir, menghasilkan efisiensi *bandwidth* yang paling optimal dan meminimalkan beban komputasi pada server (Nair 2020).

2.2 Analisis Infrastruktur Penyimpanan Data: On-Premise vs Cloud Storage

Perusahaan yang mengimplementasikan sistem absensi harus menentukan jenis arsitektur penyimpanan data di sisi server. Keputusan ini berdampak langsung pada biaya investasi, tingkat keamanan informasi, tingkat kesulitan pemeliharaan, serta skalabilitas sistem.

Tabel 2.1: Perbandingan Infrastruktur On-Premise dan Cloud Storage

Dimensi Evaluasi	**Arsitektur On-Premise**	**Arsitektur Cloud Storage**
Investasi Biaya	Biaya awal sangat tinggi untuk pembelian server fisik.	Biaya operasional rendah, berbasis sewa bulanan.
Aksesibilitas Data	Terbatas pada jaringan lokal, butuh konfigurasi VPN rumit.	Praktis, dapat diakses dari mana saja secara global.
Resiko Kehilangan Data	Tinggi jika terjadi kerusakan perangkat keras tanpa cadangan.	Rendah karena dilengkapi fitur replikasi otomatis.
Kebutuhan Tenaga Ahli	Memerlukan insinyur khusus (*maintenance engineer*).	Pemeliharaan infrastruktur diurus oleh penyedia layanan awan.
Keamanan Informasi	Sangat tinggi; kendali penuh atas data sensitif perusahaan.	Terjamin melalui protokol enkripsi standar industri.

Penyimpanan berbasis *cloud storage* menawarkan efisiensi operasional bagi perusahaan berskala menengah ke bawah karena tidak memerlukan biaya perawatan perangkat keras fisik. Namun, sistem presensi yang dibangun harus tetap mengintegrasikan penyimpanan lokal pada perangkat seluler untuk menopang kebutuhan operasional luring.

2.3 Kajian Komparatif Solusi Penyimpanan Lokal Perangkat Seluler

Penyimpanan lokal pada perangkat seluler bertindak sebagai penampung sementara bagi seluruh transaksi presensi sebelum dikirim ke database pusat. Pemilihan teknologi *database* lokal sangat memengaruhi kecepatan penulisan, kemudahan pemodelan data, serta konsumsi memori perangkat.

Isar Database dirancang secara khusus untuk ekosistem Flutter dan Dart. Karakteristik kinerjanya yang berada di tingkat sub-milidetik memastikan bahwa

Tabel 2.2: Perbandingan Solusi Database Lokal

Nama Solusi	Tipe Basis Data	Keunggulan Utama	Kelemahan Utama
SharedPreferences	Key-Value	Ringan, tanpa dependensi.	Tidak mendukung kueri kompleks.
Hive	NoSQL	Eksekusi cepat, performa tinggi.	Kueri terbatas, kurang cocok relasi.
SQLite	Relasional (SQL)	Mendukung kueri kompleks, ACID.	Memerlukan SQL manual.
Isar Database	NoSQL (Object)	Sangat cepat, tipe aman.	Ekosistem relatif baru.

perekaman data kehadiran, pengambilan koordinat lokasi, hingga pencarian log riwayat dapat berjalan secara instan tanpa mengganggu kelancaran animasi antarmuka pengguna (*UI jank*).

2.4 Penerapan Clean Architecture pada Framework Flutter

Clean Architecture yang dipopulerkan oleh Robert C. Martin merupakan sebuah pola perancangan perangkat lunak yang membagi sistem ke dalam beberapa lapisan konsentris dengan aturan ketergantungan yang ketat ke arah dalam. Implementasi pola ini dalam pengembangan aplikasi Flutter bertujuan untuk menciptakan basis kode yang modular, mudah diuji secara independen, dan adaptif terhadap perubahan teknologi. Terdapat tiga lapisan utama dalam arsitektur ini:

1. **Presentation Layer**: Lapisan terluar yang berurusan langsung dengan elemen antarmuka pengguna (*UI Widgets*) dan penangan logika tampilan (*State Management*) seperti BLoC, GetX, atau Provider.

2. **Domain Layer**: Merupakan jantung dari aplikasi yang menyimpan seluruh aturan bisnis inti. Lapisan ini bersifat independen dari kerangka kerja Flutter. Komponen di dalamnya meliputi Kasus Penggunaan (*Use Cases*), Entitas (*Entities*), dan Kontrak Repositori.

3. **Data Layer**: Bertanggung jawab atas penyediaan data ke lapisan domain. Lapisan ini mengimplementasikan kontrak repositori dan mengelola interaksi dengan sumber data eksternal (API) maupun internal (Local DB).

Melalui pemisahan tanggung jawab yang jelas ini, pengujian unit (*unit testing*) dapat dilakukan secara optimal karena setiap lapisan dapat diuji secara terisolasi menggunakan teknik simulasi objek (*mocking*).

2.5 Flutter dan Dart

Flutter merupakan kerangka kerja (*framework*) lintas platform yang dikembangkan oleh Google, dirilis secara stabil pada tahun 2018. Flutter memungkinkan pengembang untuk membuat aplikasi Android, iOS, web, dan desktop berkinerja tinggi hanya dengan satu basis kode tunggal (Google 2026). Berbeda dengan kerangka kerja lainnya, Flutter tidak menggunakan komponen bawaan sistem operasi (*OEM widgets*), melainkan me-render seluruh elemen antarmuka secara mandiri (*widget tree*) menggunakan mesin grafis Skia, sehingga menghasilkan konsistensi tampilan di berbagai platform dan mendukung kompilasi *Ahead-of-Time* (AOT) untuk kecepatan eksekusi. Bahasa pemrograman yang mendasari Flutter adalah Dart, sebuah bahasa berorientasi objek yang mendukung *null-safety*, pemrograman asinkron (*async/await*), dan pengetikan yang kuat (*strong typing*), menjadikannya sangat relevan untuk membangun logika sinkronisasi data yang kompleks (Google 2026).

2.6 PocketBase sebagai Backend-as-a-Service

Dalam ekosistem aplikasi *mobile*, ketersediaan layanan *backend* yang tangguh dan ringan sangat krusial. PocketBase merupakan solusi *Backend-as-a-Service* (BaaS) sumber terbuka yang dibangun menggunakan bahasa Go, didesain untuk didistribusikan sebagai satu file biner tunggal tanpa memerlukan dependensi yang rumit (PocketBase 2026). PocketBase menyediakan fitur *REST API* secara otomatis berdasarkan skema basis data, sistem autentikasi bawaan yang aman, pengelolaan penyimpanan berkas (*file storage*) yang terintegrasi, dan kapabilitas *real-time subscriptions*. Alasan utama pemilihan PocketBase dibandingkan layanan *cloud* komersial seperti Firebase adalah kebebasannya untuk di-*host* secara mandiri (*self-hosted*), yang memberikan kontrol penuh atas privasi data perusahaan dan memungkinkan konfigurasi penyimpanan objek tambahan melalui S3 Minio

(PocketBase 2026).

2.7 GetX sebagai State Management

Pengelolaan status (*state management*) merupakan aspek vital dalam aplikasi yang interaktif, terutama untuk menangani pembaruan status sinkronisasi luring dan daring. GetX adalah pustaka Flutter yang sangat ringan dan kuat untuk manajemen status, injeksi ketergantungan (*dependency injection*), dan manajemen rute secara reaktif (Law 2026). Berbeda dengan pola Provider atau BLoC yang membutuhkan banyak kode berulang (*boilerplate*), GetX menawarkan paradigma reaktif berbasis variabel `Rx` (misalnya `RxBool`, `RxString`) yang secara otomatis akan memperbarui elemen antarmuka ketika nilai variabel tersebut berubah. Pendekatan ini meningkatkan performa secara signifikan karena hanya komponen *widget* yang relevan yang akan dimuat ulang.

2.8 Sistem Presensi dan Validasi Geofencing

Presensi digital merupakan evolusi dari pencatatan kehadiran tradisional, di mana validasi fisik digantikan oleh parameter lokasi (*geolocation*) dan bukti foto (*biometrik/visual*). Untuk mencegah manipulasi kehadiran (seperti penggunaan lokasi palsu atau absen di luar area kerja), aplikasi melakukan validasi koordinat GPS menggunakan *geofencing*, sebuah batas virtual berdasarkan koordinat geografis. Perhitungan jarak antara posisi pengguna saat menekan tombol presensi dan koordinat pusat wilayah kantor dihitung secara luring menggunakan formula Haversine (Ismail dkk. 2023):

$$d = 2r \arcsin\left(\sqrt{\sin^2\left(\frac{\phi_2 - \phi_1}{2}\right) + \cos(\phi_1)\cos(\phi_2)\sin^2\left(\frac{\lambda_2 - \lambda_1}{2}\right)}\right) \tag{2.1}$$

Di mana:

- d menyatakan jarak aktual antara kedua titik koordinat dalam satuan meter.
- r menyatakan jari-jari bumi rata-rata.

- ϕ melambangkan garis lintang (*latitude*).
- λ melambangkan garis bujur (*longitude*).

2.9 Metode Pengembangan Agile Scrum

Penelitian ini menggunakan kerangka kerja *Agile Scrum*, sebuah metodologi pengembangan perangkat lunak adaptif yang mengedepankan kolaborasi tim dan iterasi berkesinambungan. Siklus pengembangan dibagi menjadi fase-fase pendek yang disebut *sprint*, biasanya berdurasi dua hingga empat minggu (Sinatria dkk. 2023). Pendekatan ini sangat cocok untuk pengembangan aplikasi *offline-first* yang kompleks karena memungkinkan tim pengembang untuk memprioritaskan fitur inti (seperti basis data lokal dan antarmuka presensi) di awal iterasi, dan kemudian menyempurnakan fitur pendukung (seperti algoritma sinkronisasi) di iterasi berikutnya berdasarkan evaluasi secara berkala.

2.10 Penelitian Terdahulu

Sebagai landasan untuk menemukan kebaruan (novelty) dan mencegah duplikasi, kajian terhadap beberapa penelitian sebelumnya yang relevan telah dilakukan. Beberapa studi terdahulu membahas mengenai arsitektur perangkat lunak, mode luring, serta metode presensi berbasis lokasi.

Devina et al. (2026) mengembangkan sistem informasi arisan berbasis Flutter dengan arsitektur *offline-first* menggunakan *database* awan Firebase Firestore (Devina dkk. 2026). Penelitian tersebut menunjukkan keberhasilan penerapan sinkronisasi otomatis, namun memiliki perbedaan mendasar dengan penelitian ini yang tidak menggunakan layanan *cloud* berbayar, melainkan PocketBase yang di-*host* secara mandiri.

Nadya dan Lidya (2025) merancang aplikasi absensi berbasis QR Code dengan mode *offline* dan sinkronisasi data menggunakan metode *Rapid Application Development* (RAD) (Indah dkk. 2025). Meskipun berhasil memecahkan masalah koneksi internet saat presensi, validasi yang digunakan hanyalah pindai kode QR tanpa adanya pembatasan lokasi geografis yang kuat. Penelitian ini memperbaiki

celah tersebut dengan mengintegrasikan GPS Geofencing.

Pada aspek struktural, Sinatria et al. (2023) (Sinatria dkk. 2023) dan William et al. (2024) (Laksono dkk. 2024) sama-sama mengimplementasikan *Clean Architecture* pada framework Flutter untuk meningkatkan *maintainability* aplikasi. William et al. menggunakan pola BLoC sebagai pengelola status, sedangkan penelitian ini mengadopsi GetX yang lebih ringan. Keduanya juga tidak membahas permasalahan transmisi data asinkron pada arsitektur *offline-first*. Serupa dengan itu, Azziqra dan Nuryasin (2024) mengukur tingkat rawatan arsitektur bersih pada aplikasi Al-Quran, dan memperoleh skor pemeliharaan sebesar 79% yang membuktikan keandalan arsitektur ini untuk sistem berjangka panjang (Azziqra dkk. 2024).

Dari sisi teknologi spasial, Ismail dan Zakaria (2023) merancang aplikasi presensi seluler menggunakan algoritma Haversine untuk memvalidasi jarak presensi karyawan terhadap titik kantor (Ismail dkk. 2023). Namun, sistem yang dibangun sangat bergantung pada ketersediaan koneksi internet (*online-first*) sehingga rentan gagal di area dengan sinyal lemah.

Tabel 2.3: Matriks Perbandingan Penelitian Terdahulu

No	Penulis (Tahun)	Judul	Framework	Database	Offline	Geofence
1	Devina et al. (2026)	Offline-First Mobile Grocery Arisan...	Flutter	Firebase	✓	×
2	Nadya & Lidya (2025)	Pengembangan Mode Offline... Absensi	-	Local DB	✓	×
3	Sinatria et al. (2023)	Penerapan Clean Architecture... Flutter	Flutter	-	×	×
4	William et al. (2024)	Implementasi Clean Architecture... Flutter	Flutter	Local DB	×	×
5	Azziqra & Nuryasin (2024)	Clean Architecture Aplikasi Al-Quran	Flutter	-	×	×

Berlanjut ke halaman berikutnya

Tabel 2.3 – *Lanjutan dari halaman sebelumnya*

No	Penulis (Tahun)	Judul	Framework	Database	Offline	Geofence
6	Ismail & Zakaria (2023)	Presensi Mobile Haversine	Mobile	-	×	✓
7	**Penelitian ini (2026)**	**Presensi Karyawan Offline-First**	**Flutter**	**Isar**	✓	✓

Berdasarkan perbandingan di atas, celah penelitian (*research gap*) yang diisi oleh studi ini adalah belum adanya penelitian yang menggabungkan Flutter, Isar Database, arsitektur *offline-first*, dan validasi GPS Geofencing dalam satu sistem presensi karyawan yang mandiri. Selain itu, mekanisme sinkronisasi pada penelitian ini menggunakan *SyncQueue* dengan *Cascade Sync* dan *Exponential Backoff* yang lebih canggih daripada pengiriman data tertunda konvensional.

BAB III
METODOLOGI PENELITIAN DAN PERANCANGAN SISTEM

3.1 Teknik Pengumpulan Data

Pengumpulan data untuk mendukung perancangan sistem dilakukan melalui beberapa pendekatan:

1. **Observasi**: Mengamati secara langsung kondisi jaringan dan proses presensi di lingkungan kerja PT Semeru Agung Mandiri untuk mengidentifikasi area yang minim sinyal (*blank spot*).

2. **Wawancara**: Melakukan tanya jawab terstruktur dengan tim HRD mengenai alur kerja presensi, kendala pada sistem sebelumnya, serta fitur yang dibutuhkan untuk pengelolaan absensi yang lebih baik.

3. **Studi Pustaka**: Mengumpulkan referensi dari jurnal penelitian terdahulu, buku akademis, serta dokumentasi resmi teknologi yang digunakan seperti Flutter, Isar Database, dan PocketBase.

3.2 Spesifikasi Kebutuhan Sistem

Tahap analisis kebutuhan fungsional mendefinisikan fungsionalitas spesifik yang harus diimplementasikan ke dalam aplikasi presensi seluler. Kebutuhan fungsional dikelompokkan berdasarkan hak akses pengguna aplikasi, yaitu Karyawan (Pengguna Lapangan) dan Admin HRD (Pengelola Data).

3.3 Kebutuhan Non-Fungsional

Kebutuhan non-fungsional menjelaskan kriteria kualitas operasi sistem. Berikut adalah spesifikasi kebutuhan non-fungsional pada sistem ini:

1. **NF-01 (Kinerja)**: Aplikasi harus responsif, dengan waktu muat antarmuka kurang dari 2 detik.

Tabel 3.1: Kebutuhan Fungsional Sistem

Aktor Pengguna	Kode	Deskripsi Kebutuhan Fungsional
Karyawan	F-01	Pengguna dapat melakukan login menggunakan akun yang diberikan oleh admin.
	F-02	Pengguna dapat melakukan transaksi presensi *Check-In* (Masuk).
	F-03	Pengguna dapat melakukan transaksi presensi *Check-Out* (Pulang).
	F-04	Pengguna dapat melakukan presensi dalam kondisi luring (tanpa internet).
	F-05	Aplikasi memvalidasi lokasi presensi pengguna menggunakan teknologi GPS Geofencing.
	F-06	Pengguna dapat melihat riwayat kehadiran pribadi secara *real-time*.
Admin HRD	F-07	Admin dapat mengelola akun karyawan.
	F-08	Admin dapat memantau lokasi kehadiran karyawan melalui *dashboard*.
	F-09	Admin dapat mengekspor laporan rekapitulasi presensi bulanan ke dalam format Excel.

2. **NF-02 (Keandalan)**: Data presensi harus dijamin tersimpan secara aman di database lokal ketika tidak ada koneksi internet.

3. **NF-03 (Otomatisasi)**: Sistem harus dapat melakukan sinkronisasi data secara otomatis di latar belakang saat koneksi internet pulih tanpa perlu intervensi manual dari pengguna.

4. **NF-04 (Portabilitas)**: Aplikasi minimal mendukung sistem operasi Android versi 5.0 (API level 21) atau yang lebih baru.

5. **NF-05 (Keamanan)**: Komunikasi data antara aplikasi seluler dan server diamankan menggunakan token otorisasi *JSON Web Token* (JWT).

3.4 Metodologi Pengembangan Scrum dan Perencanaan Sprint

Sistem presensi dikembangkan menggunakan metodologi rekayasa perangkat lunak *Agile* dengan kerangka kerja *Scrum*. Metodologi ini membagi proses pengembangan ke dalam iterasi jangka pendek yang disebut *sprint*. Peran utama dalam tim Scrum meliputi *Product Owner* (memiliki visi produk dan mengelola

prioritas *backlog*), *Scrum Master* (memandu proses dan menghilangkan hambatan teknis), dan *Development Team* (tim pengembang perangkat lunak).

Seluruh siklus pengembangan dipetakan ke dalam 5 *sprint* berturut-turut, di mana durasi masing-masing *sprint* ditetapkan selama 2 minggu. Manajemen *backlog* dan visualisasi kemajuan tugas dikelola secara digital menggunakan platform Jira Board.

Tabel 3.2: Perencanaan Sprint

Sprint	**Fokus Utama**	**Detail Item Pekerjaan (Backlog)**
Sprint 1	Arsitektur & Database	Inisialisasi proyek Flutter, penerapan Clean Architecture, pembuatan skema Isar.
Sprint 2	Autentikasi & Core UI	Pembuatan antarmuka login, integrasi API, penyimpanan token.
Sprint 3	Fitur Presensi Luring	Pembuatan antarmuka presensi, GPS, pembatas Geofencing.
Sprint 4	Sinkronisasi Latar Belakang	Implementasi pendeteksi jaringan, konfigurasi WorkManager.
Sprint 5	Pelaporan & Pengujian	Pembuatan antarmuka admin, fungsi ekspor Excel, pelaksanaan UAT.

3.5 Perancangan Skema Database Isar

Dalam arsitektur *offline-first*, rancangan tabel data lokal harus mampu menampung seluruh detail transaksi presensi serta menyediakan bendera penanda status sinkronisasi. Berikut adalah representasi model data `AttendanceLocal` dalam bentuk struktur koleksi Isar:

```
@collection
class AttendanceLocal {
  Id id = Isar.autoIncrement;

  // ID dari server (PocketBase). Bernilai null jika data belum disinkronka
  String? odId;

  late String userId;
```

```
  late DateTime checkInTime;
  DateTime? checkOutTime;

  // Koordinat lokasi presensi
  double? lat, lng;

  // Bendera penanda sinkronisasi offline-first
  bool isSynced = false;
  bool isOfflineEntry = false;
  late String deviceId;
}
```

Anotasi unik pada atribut menjamin bahwa setiap transaksi presensi memiliki pengidentifikasi unik universal (UUID). Flag `isSynced` memegang peran sentral dalam menentukan apakah data tersebut sudah berhasil dikirim ke *server* atau masih perlu dimasukkan ke dalam antrean sinkronisasi (*SyncQueue*). Hal ini mencegah terjadinya duplikasi pencatatan kehadiran di sisi server apabila proses sinkronisasi mengalami kegagalan transmisi jaringan di tengah jalan.

3.6 Algoritma Validasi Geofencing dan Sinkronisasi Data

Keandalan sistem presensi luring ditopang oleh dua mekanisme utama: validasi batas geofencing tanpa koneksi internet dan sinkronisasi data asinkron latar belakang.

3.6.1 Validasi Jarak Geofencing Lokal

Untuk mencegah manipulasi kehadiran (seperti penggunaan lokasi palsu atau absen di luar area kerja), aplikasi melakukan validasi koordinat GPS langsung di dalam perangkat luring. Perhitungan jarak antara posisi pengguna saat menekan tombol presensi dan koordinat pusat wilayah kantor dihitung menggunakan formula Haversine, sebagaimana telah diuraikan pada Bab 2 Tinjauan Pustaka.

3.6.2 Mekanisme Sinkronisasi Asinkron

Proses sinkronisasi data presensi bekerja secara asinkron menggunakan pustaka WorkManager dan pengamat status koneksi internet (*Connectivity Observer*). Ketika koneksi terdeteksi terputus, transaksi presensi tetap disimpan di Isar dengan atribut `isSynced = false`. Saat perangkat mendeteksi kembalinya jaringan internet, WorkManager akan diaktifkan untuk mengekstrak seluruh baris data dari Isar dengan filter `isSynced == false`, mengompresinya menjadi *payload* JSON, dan mengirimkannya ke API server menggunakan skema retri *exponential backoff*.

3.7 Arsitektur Infrastruktur Sistem

Secara konseptual, sistem yang akan dikembangkan menggunakan arsitektur tiga tingkat (*three-tier architecture*) modern. Lapis pertama adalah perangkat seluler (klien) yang menjalankan kerangka kerja Flutter dan menampung data menggunakan Isar. Lapis kedua adalah jaringan transmisi (internet), yang menjembatani klien dengan lapis ketiga, yaitu server *backend*. Pada lapis server, sistem dikelola oleh PocketBase yang diimplementasikan pada peladen virtual (*Virtual Private Server/VPS*). Khusus untuk berkas multimedia seperti foto bukti kehadiran, infrastruktur server diintegrasikan dengan S3 Minio, yang bertindak sebagai objek penyimpanan persisten berskala besar (*object storage backend*). Aliran data transaksi dimulai dari perekaman foto dan lokasi pengguna, diteruskan ke *SyncQueue* lokal, dan akhirnya diproses oleh antarmuka REST API PocketBase.

3.8 Perancangan Antarmuka (Wireframe)

Perancangan antarmuka aplikasi difokuskan pada fungsionalitas intuitif yang memungkinkan perekaman kehadiran secara luring. *(Gambar wireframe untuk halaman login, dasbor karyawan, halaman presensi, riwayat, dan dasbor admin akan dilampirkan oleh peneliti pada laporan akhir)*

DAFTAR PUSTAKA

Azziqra, M. Zihad and Ilyas Nuryasin (Dec. 2024). "Implementasi Clean Architecture pada Aplikasi Mobile Al-Quran Berbasis Flutter". In: *JOISIE: Journal Of Information Systems And Informatics Engineering* 8.2, pp. 369–380. ISSN: 2527-3116. DOI: 10.35145/joisie.v8i2.4763.

Devina dkk. (July 2026). "Design and Implementation of an Offline-First Mobile Grocery Arisan Information System Using Flutter and Firebase". In: *NUANSA INFORMATIKA* 20.2, pp. 133–148. DOI: 10.25134/ilkom.v20i2.582. URL: https://journal.fkom.uniku.ac.id/ilkom/article/view/582.

Google (2026). "Flutter Documentation". Diakses pada 21 Juli 2026. URL: https://docs.flutter.dev.

HashMicro (2024). "Kelemahan Sistem Presensi Tradisional dan Solusinya". Diakses pada 21 Juli 2026. URL: https://www.hashmicro.com.

Indah, Nadya Kusuma and Lidya Wati (Aug. 2025). "Pengembangan Mode Offline dan Sinkronisasi Data pada Aplikasi Absensi Berbasis QR Code untuk Karyawan PT. Bintang Energi Pratama". In: *JUTITI: Jurnal Teknik Informatika dan Teknologi Informasi* 5.2, pp. 417–427. ISSN: 2827-9387. DOI: 10.55606/jutiti.v5i2.5544.

Ismail and Hadi Zakaria (Aug. 2023). "Rancang Bangun Aplikasi Presensi Berbasis Mobile dengan Metode Haversine". In: *LOGIC: Jurnal Ilmu Komputer dan Pendidikan* 1.5, pp. 1355–1369. ISSN: 2985-4172. URL: https://journal.mediapublikasi.id/index.php/logic.

Laksono, William Prapdeson dkk. (Jan. 2024). "Implementasi Clean Architecture Dalam Membangun Aplikasi Mobile Menggunakan Flutter". In: *Nusantara Journal of Multidisciplinary Science* 1.6, pp. 173–180. URL: https://jurnal.intekom.id/index.php/njms/article/view/264.

Law, Jonatas (2026). "GetX Documentation". Diakses pada 21 Juli 2026. URL: https://pub.dev/packages/get.

Mekari Talenta (2024). "Sistem Kehadiran Karyawan: Panduan Lengkap". Diakses pada 21 Juli 2026. URL: https://www.talenta.co.

Nair, et al. (2020). “Empirical Evaluation of Synchronization Strategies for Offline-First Mobile Applications”. In: *IRE Journals*. URL: https://irejournals.com.

PocketBase (2026). “PocketBase Documentation”. Diakses pada 21 Juli 2026. URL: https://pocketbase.io/docs.

Politeknik Negeri Bengkalis (2024). “Sistem Presensi Karyawan Berbasis Mobile”. Diakses pada 21 Juli 2026. URL: https://polbeng.ac.id.

ResearchGate (2024). “Offline-first Software Architecture for Mobile Applications”. In: *ResearchGate*. URL: https://www.researchgate.net.

Sinatria, Mohammad Bijantium, Oman Komarudin, and Kamal Prihandani (May 2023). “Penerapan Clean Architecture dan SOLID Principle dalam Membangun Aplikasi Berbasis Mobile dengan Menggunakan Framework Google Flutter”. In: *INFOTECH Journal* 9.1, pp. 132–146. DOI: 10.31949/infotech.v9i1.5237. URL: https://ejournal.unma.ac.id/index.php/infotech/article/view/5237.

SmartTech Devs (2024). “Offline-First Mobile App Development”. Diakses pada 21 Juli 2026. URL: https://smarttechdevs.com.

Stackademic (2024). “Solving The Basement Problem with Offline-First Architecture”. Diakses pada 21 Juli 2026. URL: https://stackademic.com.